Pocket SDR Command Reference

ver.0.20 2026-08-07

Table of Contents

- **Signal ID**
- **pocket_scan** - Scan and list USB devices
- **pocket_conf** - Configure Pocket SDR FE device registers
- **pocket_dump** - Capture digital IF data from Pocket SDR FE
- **pocket_acq** - GNSS signal acquisition from IF data
- **pocket_trk** - GNSS signal tracking, nav decoding, and PVT generation
- **pocket_web** - GNSS receiver server with Web UI
- **pocket_snap** - Snapshot positioning from IF data
- **pocket_calib** - Antenna array attitude and per-CH bias calibration
- **convbin** - Convert receiver logs / RTCM streams to RINEX
- **str2str** - Stream converter / forwarder (RTKLIB-derived)

Signal ID

GNSS signals supported by Pocket SDR and the corresponding string IDs used with the `-sig` option of `pocket_trk`, `pocket_acq`, `pocket_snap`, and related Python scripts.

Table 1. Pocket SDR Signal IDs

System	Signal	Signal ID
GPS	L1C/A	`L1CA`
	L1C-D	`L1CD`
	L1C-P	`L1CP`
	L2C-M	`L2CM`
	L5-I	`L5I`
	L5-Q	`L5Q`
GLONASS	L1C/A (L1OF)	`G1CA`
	L2C/A (L2OF)	`G2CA`
	L1OCd	`G1OCD`
	L1OCp	`G1OCP`
	L2OCp	`G2OCP`
	L3OCd	`G3OCD`
	L3OCp	`G3OCP`
Galileo	E1-B	`E1B`
	E1-C	`E1C`
	E5a-I	`E5AI`
	E5a-Q	`E5AQ`
	E5b-I	`E5BI`
	E5b-Q	`E5BQ`
	E5 AltBOC (Q)	`E5ABQ`
	E6-B	`E6B`
	E6-C	`E6C`
QZSS	L1C/A	`L1CA`
	L1C/B	`L1CB`
	L1C-D	`L1CD`
	L1C-P	`L1CP`
	L1S	`L1S`
	L2C-M	`L2CM`

System	Signal	Signal ID
	L5-I	`L5I`
	L5-Q	`L5Q`
	L5S-I	`L5SI` / `L5SIV` [*1]
	L5S-Q	`L5SQ` / `L5SQV` [*1]
	L6D	`L6D`
	L6E	`L6E`
BeiDou	B1I	`B1I`
	B1C-D	`B1CD`
	B1C-P	`B1CP`
	B2a-D	`B2AD`
	B2a-P	`B2AP`
	B2I	`B2I`
	B2b-I	`B2BI`
	B3I	`B3I`
NavIC	L1-SPS-D	`I1SD`
	L1-SPS-P	`I1SP`
	L5-SPS	`I5S`
	S-SPS	`ISS` [*2]
SBAS	L1C/A	`L1CA`
	L5-I	`L5I`
	L5-Q	`L5Q`

[*1] QZSS L5S verification mode signals. [*2] Pocket SDR FE currently does not support S-band signals; `ISS` is provided for software-only use with externally captured IF data.

pocket_scan - Scan and list USB devices

Synopsis

```
pocket_scan [-e]
```

Description

Scan and list USB devices visible to the host. On Windows the Cypress CyAPI library is used; on Linux/macOS libusb-1.0 is used. The output lists USB address/bus/port, link speed, VID/PID and a device-name string.

Options

- `-e`
  - Show endpoint information (interface, alt-setting, endpoint, direction, max packet size) for each device.

pocket_conf - Configure Pocket SDR FE device registers

Synopsis

```
pocket_conf [-s] [-a] [-h] [-p bus[,port]] [-v] [conf_file]
```

Description

Configure or show the MAX2771 register settings of a Pocket SDR FE device. If `conf_file` is given, the settings in the configuration file are written to the device registers. The configuration file is a text file containing MAX2771 register field settings in either keyword=value form or hexadecimal form. Comments after `#` are ignored. If `conf_file` is omitted, the command prints the current device settings in the same format.

Keyword=value form:

```
[CHx]
FCEN     = 97  # ...
FBW      =  0  # ...
F3OR5    =  1  # ...
...
```

Hexadecimal form:

```
#CH  ADDR       VALUE
  1  0x00  0xA2241C17
  1  0x01  0x20550288
...
```

Options

- `-s`
  - Save the settings to the device EEPROM. Saved settings are also loaded automatically at device reset.
- `-a`
  - Show all register fields (including reserved bits).
- `-h`
  - Read or write registers in hexadecimal form.

- `-p bus[,port]`
  - USB bus and port number of the target Pocket SDR FE device. Without this option, the first device found is selected.
- `-v`
  - Print version and exit.
- `conf_file`
  - Path of the configuration file. Without this argument, the current register field settings of the device are printed.

References

[1] Maxim Integrated, MAX2771 Multiband Universal GNSS Receiver, July 2018.

pocket_dump - Capture digital IF data from Pocket SDR FE or SoapySDR device

---

Synopsis

```
pocket_dump [-t tsec] [-r] [-p bus[,port]] [-c conf_file] [-q] [-v]
            [-driver name] [-fmt {CS8|CS16}] [-f fs] [-fo freq[,...]]
            [-gain gain] [-bw bw] [-gpio val] [-NO_SYNC]
            [path [path ...]]
```

Description

Capture and dump digital IF (DIF) data from a Pocket SDR FE device or a SoapySDR-supported device. Pocket SDR FE capture writes one output file per RF channel unless `-r` is specified. SoapySDR capture writes one complex sample stream to the first output path; with multiple RF channels the channels are written channel-planar in blocks of a processing cycle, which `pocket_trk` replays as a multi-channel input. Capturing stops on the configured duration (`-t`), or by Ctrl-C. A `<path>.tag` companion file is written for each output, recording the IF format, sampling rate, LO frequencies, and per-CH sampling type / bit width so downstream tools (e.g. `pocket_trk` , `pocket_acq`) can auto-configure.

Options

- `-t tsec`
  - Capture duration in seconds. `0` or omitted: capture until Ctrl-C.
- `-r`
  - Dump the raw bit-packed device output without per-channel separation or quantization. With `-r` , only the first output path is used.
- `-p bus[,port]`
  - USB bus and port number of the target device. Without this option, the first device found is selected.
- `-c conf_file`
  - Configure the device with a `pocket_conf` -format file before capture starts.
- `-driver name`
  - Use a SoapySDR device instead of a Pocket SDR FE device. Examples: `lime` , `uhd` , `bladerf` , `rtlsdr` , `plutosdr` . Driver-specific arguments accepted by SoapySDR can also be passed. Multiple devices are specified as specs separated by `;` , and `nchan=<n>` in a spec selects the number of RX channels of the device (default 1), up to 8 RF channels in total. With multiple devices the external clock and time sources are selected and the

streams are started at a common time latched by a 1PPS, so a common 10 MHz and 1PPS have to be distributed to the devices.

    - `-driver "driver=uhd,serial=31A1;driver=uhd,serial=31B5"`
    - `-driver "driver=uhd,serial=31A1,nchan=2"`

- `-fmt {CS8|CS16}`
  - SoapySDR sample format. [`CS8`]
- `-f fs`
  - SoapySDR sampling rate in MHz. [`12`]
- `-fo freq[,...]`
  - SoapySDR RF center frequency of each RF channel in MHz. This option is required with `-driver`. A channel with an omitted or `0` frequency uses the frequency of the first channel.
- `-gain gain`
  - SoapySDR RF gain in dB. Omitted or `0`: use automatic gain if the driver supports it.
- `-bw bw`
  - SoapySDR RF bandwidth in MHz. Omitted or `0`: leave driver default.
- `-gpio val`
  - Output `val` (0-255) to the GPIO pins of the SoapySDR device (USRP: front-panel GPIO). The pins are set to outputs at the device open.
- `-NO_SYNC`
  - With multiple SoapySDR devices, do not synchronize the device times by a 1PPS. The devices run on their own clocks and the RF channels of different devices are NOT aligned in samples.
- `-q`
  - Suppress the runtime status display.
- `-v`
  - Print version and exit.
- `[path [path ...]]`
  - Output file paths. The k-th path is the file for CH(k); trailing paths can be omitted (those CHs are not written). With `-r`, only the first path is used.
  - `""` discards the data for that CH; `-` writes to stdout (binary mode on Windows).
  - If all paths are omitted, the default paths are:
    - `chN_YYYYMMDD_hhmmss.bin` for each CH N (UTC start date/time).

pocket_acq - GNSS signal acquisition from IF data

Synopsis

```
pocket_acq [-sig sig] [-prn prn[,...]] [-tint tint] [-toff toff]
           [-fmt {INT8|INT8X2|CS8|CS16}] [-f freq] [-fi freq]
           [-d freq[,freq]] [-nz]
           [-w file] [-v] file
```

Description

Search GNSS signals in a digital IF data file and report acquisition results (Doppler, code offset, C/N0). For each PRN, parallel-code FFT search with non-coherent integration is performed.

If a tag file `<file>.tag` exists alongside the input, the IF format, sampling frequency, LO frequencies, and per-CH sampling type are taken from the tag and the corresponding command-line options are ignored.

Options ([]: default)

- `-sig sig`
  - GNSS signal type ID (`L1CA`, `L2CM`, `L5I`, ...). For supported IDs see `doc/signal_IDs.pdf`. [`L1CA`]
- `-prn prn[,...]`
  - PRN numbers, comma-separated. A range like `1-32` is accepted. For GLONASS FDMA signals (`G1CA`, `G2CA`) this is the FCN. [`1`]
- `-tint tint`
  - Coherent integration time in ms. [code cycle]
- `-toff toff`
  - Time offset from the start of the IF data in ms. [`0.0`]
- `-fmt {INT8|INT8X2|CS8|CS16}`
  - IF data format. `INT8` is single-channel real int8, `INT8X2` is single-channel interleaved int8 IQ, and `CS8`/`CS16` are SoapySDR complex int8/int16 formats. Pocket SDR FE packed RAW formats are not expanded by `pocket_acq`; dump channel-separated files with `pocket_dump` first. [`INT8X2`]
- `-f freq`
  - Sampling frequency of the IF data in MHz. [`12.0`]
- `-fi freq`

    - IF frequency in MHz. `0` means IQ (zero-IF) sampling. [`0.0`]
- `-d ref_dop[,max_dop]`
    - Reference and maximum Doppler frequency to search in Hz. [`0.0,5000.0`]
- `-nz`
    - Disable zero-padding for circular correlation. [enabled]
- `-w file`
    - FFTW wisdom file. [`../python/fftw_wisdom.txt`]
- `-v`
    - Print version and exit.
- `file`
    - Input digital IF data file. The format is a series of `int8_t` (I-sampling), interleaved `int8_t` (IQ-sampling), or SoapySDR complex samples for `CS8` / `CS16`, unless overridden by the tag file.

pocket_trk - GNSS signal tracking, nav decoding, and PVT generation

Synopsis

```
pocket_trk [-sig sig -prn prn[,...] [-rfch ch[,...]] ...]
           [-fmt {INT8|INT8X2|RAW8|RAW16|RAW16I|RAW32|CS8|CS16}]
           [-f freq] [-fo freq[,...]] [-IQ {1|2}[,...]] [-bits {2|3}[,...]]
           [-toff toff] [-tscale scale] [-ti tint]
           [-p bus[,port]] [-c conf_file]
           [-driver name] [-gain gain] [-bw bw] [-gpio val] [-EXT_CLOCK]
           [-NO_SYNC] [-fd dopp] [-log path] [-nmea path] [-rtcm path] [-raw
path] ...
           [-arch nch] [-geom file] [-sens name[,...]]
           [-h height] [-opt file] [-debug file] [-v] [file]
```

Description

Search and track GNSS signals in input digital IF data, extract observation data, decode navigation data, and generate PVT solutions. Observation and navigation data can be streamed as RTCM3, PVT solutions as NMEA, and raw IF data and event logs can be streamed independently. The output stream options `-log`, `-nmea`, `-rtcm`, and `-raw` may each be specified multiple times, up to 8 output streams in total; each occurrence opens an additional stream of that type (e.g., `-nmea sol.nmea -nmea :2001` outputs NMEA to a file and a TCP server simultaneously).

The input can be a local file, a TCP stream, a Pocket SDR FE device, or a SoapySDR-supported device (LimeSDR, HackRF, RTL-SDR, ...). If the file path is omitted and `-driver` is not specified, the input is taken from a Pocket SDR FE device directly. For file inputs, if `<file>.tag` exists the IF format, sampling rate, LO frequencies, and per-CH sampling types are auto-configured from the tag, and the corresponding `-fmt`, `-f`, `-fo`, and `-IQ` options are ignored.

Options ([]: default)

- `-sig sig -prn prn[,...] [-rfch ch[,...]] ...`
  - GNSS signal type ID (`L1CA`, `L2CM`, ...) and PRN list. PRNs are comma-separated and may use ranges (e.g. `1-32`). For GLONASS FDMA signals (`G1CA`, `G2CA`), the PRN is treated as the FCN.
  - `-rfch` assigns the signal to specific RF channel(s) (comma- or `-`-separated). Without `-rfch`, the RF channel is auto-selected. The whole `-sig`/`-prn`/`-rfch` triple may be repeated to track multiple signal types.
- `-fmt {INT8|INT8X2|RAW8|RAW16|RAW16I|RAW32|CS8|CS16}`

    - Input IF data format. `INT8` = int8 (I), `INT8X2` = interleaved int8 (IQ), `RAW8/16/32` = Pocket SDR FE 2/4/8CH packed raw, `RAW16I` = Spider SDR FE packed raw (8CH), `CS8` / `CS16` = SoapySDR complex int8/int16. [`INT8X2`]
- `-f freq`
    - Sampling frequency of the IF data in MHz. [`12.0`]
- `-fo freq[,...]`
    - LO frequency for each RF channel in MHz, comma-separated. With `RAW8/16/32`, 2/4/8 frequencies are required. With `-driver`, these are the SoapySDR RF center frequencies of the RF channels and the first one is required; a channel with an omitted or `0` frequency uses the first one.
- `-IQ {1|2}[,...]`
    - Per-RF-CH sampling type: `1` = I, `2` = IQ. Used with `RAW8/16/32`. [`2,2,2,2,2,2,2,2`]
- `-bits {2|3}[,...]`
    - Per-RF-CH sample bit width (2 or 3). Used with `RAW8/16/32` when `-IQ` is `1` (I-sampling). [`2,2,2,2,2,2,2,2`]
- `-toff toff`
    - Time offset from the start of the IF data in s. [`0.0`]
- `-tscale scale`
    - Replay time scale for file inputs. [`1.0`]
- `-ti tint`
    - Update interval of the runtime tracking status in s. `0` suppresses the status display. [`0.1`]
- `-p bus[,port]`
    - USB bus and port number of the Pocket SDR FE device, when input is from the device.
- `-c conf_file`
    - Configure the Pocket SDR FE device with a `pocket_conf`-format file before tracking starts.
- `-driver name`
    - SoapySDR driver name (`lime`, `hackrf`, `rtlsdr`, `sdrplay`, `bladerf`, ...). Selects a SoapySDR device as the input. Multiple devices are specified as specs separated by `;`, and `nchan=<n>` in a spec selects the number of RX channels of the device (default 1), up to 8 RF channels in total. With multiple devices the external clock and time sources are selected and the streams are started at a common time latched by a 1PPS, so a common 10 MHz and 1PPS have to be distributed to the devices. The RF channels are then handled like the RF channels of a multi-CH frontend, including `-RFCH` assignment and `-arch` array combining.
- `-gain gain`
    - SoapySDR overall gain in dB.
- `-bw bw`

    - SoapySDR analog filter bandwidth in MHz.
- `-gpio val`
    - Output `val` (0-255) to the GPIO pins of the SoapySDR device (USRP: front-panel GPIO). The pins are set to outputs at the device open.
- `-EXT_CLOCK`
    - Select the external clock source of the SoapySDR device.
- `-NO_SYNC`
    - With multiple SoapySDR devices, do not synchronize the device times by a 1PPS. The devices run on their own clocks and the RF channels of different devices are NOT aligned in samples (for non-coherent uses such as a band per device).
- `-fd dopp`
    - Maximum Doppler frequency to search signals in Hz. Overrides the `max_dop` key in `-opt file` if both are given. [from `-opt file` or default `5000`]
- `-log path`
    - Output stream path for the tracking log (observation, navigation, PVT, events). May be specified multiple times to open multiple log streams (max 8 output streams in total including `-nmea`, `-rtcm`, and `-raw`). The stream path is one of:
        - `(1)` local file path without `:`. Time keywords (`%Y`, `%m`, `%d`, `%h`, `%M`) are expanded as in RTKLIB streams.
        - `(2)` `:port` (TCP server)
        - `(3)` `address:port` (TCP client)
- `-nmea path`
    - Output stream path for PVT solutions as NMEA `GNRMC`, `GNGGA`, and `GNGSV` sentences. Same path syntax and repeatability as `-log`.
- `-rtcm path`
    - Output stream path for raw observations and navigation data as RTCM3.3 messages. Same path syntax and repeatability as `-log`.
- `-raw path`
    - Output stream path for raw IF data. Same path syntax and repeatability as `-log`. Enabled only for Pocket SDR FE or SoapySDR device inputs.
- `-arch nch`
    - Number of antenna array channels. Array CHs are appended after the RF CHs. [`0`]
- `-geom file`
    - Array element positions file (body-frame, `pocket_sdr.py` array geometry format) applied at startup. [none]
- `-sens name[,...]`
    - Sensors of the RF channels of a SoapySDR device to log every 10 s as `$SENS,<time>,<rfch>,<name>,<value>`, e.g. `temp`, `rssi`, `lo_locked` (RF channel) or `ref_locked` (device) of a USRP. [`temp`]

- `-h height`
  - Console height (rows) for the runtime status display. [`64`]
- `-opt file`
  - System options file. The recognized keys and defaults are listed below; `app/pocket_trk/pocket_trk_default.conf` provides an example profile. [none]
- `-debug file`
  - Enable RTKLIB trace output to the given file (trace level 3).
- `-v`
  - Print version and exit.
- `[file]`
  - Input IF data file path. If omitted and no `-driver` is given, the input is taken from a Pocket SDR FE device. With `<file>.tag` present, format/sampling parameters are auto-configured.

System option keys

The file passed with `-opt` contains `key = value` entries. Without `-opt`, the library defaults below are used. The shipped `app/pocket_trk/pocket_trk_default.conf` is an example profile; its differing values are shown in the last column.

Key	Meaning	Library default	Example profile
`fftw_wisdom`	FFTW wisdom file path (handled by `pocket_trk`)	none	none
`epoch`	PVT epoch interval (s)	`1.0`	`1.0`
`lag_epoch`	Maximum PVT epoch lag (s)	`0.5`	`0.5`
`el_mask`	PVT elevation mask (deg)	`15`	`15`
`sp_corr`	Early/late correlator spacing (chip)	`0.25`	`0.2`
`t_acq`	Normal-acquisition integration time (s)	`0.02`	`0.02`
`t_acq_ext`	Doppler-assisted acquisition integration time (s)	`0.10`	`0.10`
`t_coh`	Requested synchronized-pilot PLL coherent interval (s)	`0.02`	`0.02`
`t_dll`	DLL integration time (s)	`0.02`	`0.02`
`b_dll`	DLL bandwidth (Hz)	`0.25`	`0.5`
`b_pll`	PLL bandwidth (Hz)	`5.0`	`10.0`
`b_fll_w`	Wide FLL bandwidth (Hz)	`5.0`	`10.0`
`b_fll_n`	Narrow FLL bandwidth (Hz)	`2.0`	`5.0`
`max_dop`	Acquisition Doppler limit (Hz)	`5000`	`10000`
`thres_cn0_l`	Normal-acquisition lock threshold (dB-Hz)	`34.0`	`34.0`
`thres_cn0_ext`	Assisted-acquisition threshold floor (dB-Hz)	`30.0`	`30.0`
`thres_cn0_u`	Normal tracking-loss threshold (dB-Hz)	`25.0`	`25.0`

Key	Meaning	Library default	Example profile
`thres_pli`	Carrier-lock indicator threshold	`0.25`	`0.25`
`lost_th`	Consecutive bad 0.5 s windows before loss	`4`	`4`
`bump_jump`	Enable BOC bump-jump (`0` / `1`)	`0`	`1`
`max_acq`	Longest code period for unassisted scheduling (ms)	`4.0`	`4.0`
`fast_acq`	Enable fast acquisition (`0` / `1`)	`0`	`0`

Assisted acquisition uses the larger of `thres_cn0_ext` and the applicable tracking-loss threshold plus 1 dB. L6 tracking uses an internal `32.5 dB-Hz` loss threshold instead of `thres_cn0_u`. Pilot coherent integration starts only after secondary-code synchronization; it falls back to per-code-period Costas updates if synchronization is lost or `b_pll * K * T >= 0.4`, where `K = max(1, round(t_coh / T))`.

`fast_acq = 1` enables fast acquisition: satellite visibility and Doppler are predicted from the host PC clock and the navigation data, almanac, and last FIX position saved in `.pocket_navdata.csv`, and satellites predicted below the horizon are skipped. The prediction assumes the signal time is close to the PC clock; when an IF data file recorded at a different time, or generated by a GNSS simulator, is replayed, the prediction can be wrong and signal acquisition may never succeed - set `fast_acq = 0` in that case.

All numeric keys other than `fftw_wisdom` are passed to `sdr_rcv_setopt()`.

pocket_web - GNSS receiver server with Web UI

Synopsis

```
pocket_web [-web [addr:]port] [-html dir] [-ini file] [-start]
           [-debug file] [-v]
```

Description

Run a GNSS receiver as a server controlled from a Web UI in a browser. Static Web UI files are served over HTTP, and commands and monitor data are exchanged over WebSocket (see `doc/design_web_ui.md`). The receiver itself provides the same signal tracking, navigation data decoding, and PVT generation as `pocket_trk`.

The AP does not start the receiver by itself unless `-start` is given. The input source, output streams, signal selection, and system options are all set from the Web UI, and the receiver is started and stopped from there. The settings are restored from `-ini file` at startup and saved when the receiver is stopped and when the AP exits, so a session continues where the previous one left off. The AP prints no runtime status of its own; the receiver is monitored from the Web UI.

The acquisition mode `Fast` of the Web UI (receiver option `-FAST_SRCH`, also selectable in `pocket_sdr.py` and by the `fast_acq` system option key of `pocket_trk`) predicts satellite visibility and Doppler from the host PC clock and the navigation data, almanac, and last fix position saved in `.pocket_navdata.csv`, assuming the signal time is close to the PC clock. When an IF data file recorded at a different time, or generated by a GNSS simulator, is replayed, the prediction can be wrong and signal acquisition may never succeed. If acquisition stalls in the `Fast` mode, switch the acquisition mode back to `Full`.

Options ([]: default)

- `-web [addr:]port`
  - TCP port of the Web UI server, and optionally the bind address. The address defaults to `127.0.0.1` (loopback only); specify `0.0.0.0:port` to accept LAN clients. The interface is unauthenticated - do not expose it to untrusted networks. Required.
- `-html dir`
  - Document root of the Web UI files. [`<exe_dir>/../html`]
- `-ini file`
  - Settings file for the receiver configuration and the system options. [`pocket_web.ini`]
- `-start`

    - Start the receiver at startup with the restored settings, as the Start command of the Web UI does. On error the AP keeps running so that the settings can be fixed and the receiver started from the Web UI. [start stopped]
- `-debug file`
    - Enable RTKLIB trace output to the given file (trace level 3).
- `-v`
    - Print the version and exit.

Example

```
pocket_web -web 0.0.0.0:8080
```

`run_sdr.sh` in the top directory starts and stops the server. On MSYS2 it stops it with SIGQUIT, which reaches a native Windows process as CTRL_BREAK_EVENT; a plain `kill` is `TerminateProcess()` and would leave the RF frontend streaming, which needs a USB reset to recover.

HTTPS with a Reverse Proxy (reference)

The Web UI has neither TLS nor authentication of its own. For access outside a trusted LAN, keep `pocket_web` bound to loopback and put a reverse proxy in front of it. The proxy only has to forward the WebSocket upgrade; everything else is plain HTTP proxying.

```
pocket_web -web 8080          # loopback only; the proxy is the only client
```

Caddy [1] issues and renews the certificate itself and forwards WebSockets without extra configuration:

```
sdr.example.com {
    reverse_proxy 127.0.0.1:8080
}
```

nginx [2] needs the upgrade headers passed through explicitly:

```
server {
    listen 443 ssl;
    server_name sdr.example.com;
    ssl_certificate     /etc/letsencrypt/live/sdr.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/sdr.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Upgrade    $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host       $host;
        proxy_read_timeout 3600s;   # the monitor WebSocket is long-lived
    }
}
```

Notes:

- Add authentication at the proxy (`auth_basic` for nginx, `basic_auth` for Caddy). TLS alone only hides the traffic; anyone reaching the proxy can still start, stop and reconfigure the receiver.
- `proxy_read_timeout` has to outlast an idle WebSocket. The default 60 s is enough while a page pushes data, but a client sitting on a page with no subscription would be disconnected.
- The browser upgrades to `wss:` by itself, since the Web UI derives the WebSocket scheme from the page URL. No option change is needed.
- A public certificate needs a public domain name. On a closed LAN, use a local CA instead, such as Caddy's internal CA (`tls internal`) or mkcert [3]; certbot [4] is the usual client for public certificates.

References:

- [1] Caddy, reverse_proxy directive, **https://caddyserver.com/docs/caddyfile/directives/reverse_proxy**
- [2] NGINX, WebSocket proxying, **https://nginx.org/en/docs/http/websocket.html**
- [3] mkcert, locally-trusted development certificates, **https://github.com/FiloSottile/mkcert**
- [4] certbot, **https://certbot.eff.org/**

pocket_snap - Snapshot positioning from IF data

Synopsis

```
pocket_snap [-ts time] [-pos lat,lon,hgt] [-ti sec] [-toff toff]
            [-f freq] [-fi freq] [-tint tint] [-sys sys[,...]]
            [-verb] [-w file] [-v] -nav file [-out file] file
```

Description

Snapshot positioning from a digitized IF data file. The command searches GNSS signals over a short integration window, resolves ms ambiguities in code offsets using a coarse position (or a Doppler-only solution if no coarse position is given), and estimates the receiver position by least-squares from the resolved code offsets.

Options ([]: default)

- `-ts time`
  - Capture start time in UTC, as `YYYY/MM/DD HH:MM:SS`. [parsed from the file name]
- `-pos lat,lon,hgt`
  - Coarse receiver position (latitude/longitude in deg, height in m). [no coarse position]
- `-ti sec`
  - Time interval between snapshots in s. `0.0` = single snapshot. [`0.0`]
- `-toff toff`
  - Time offset from the start of the IF data in s. [`0.0`]
- `-f freq`
  - Sampling frequency of the IF data in MHz. [`12.0`]
- `-fi freq`
  - IF frequency in MHz. `0` = IQ (zero-IF). [`0.0`]
- `-tint tint`
  - Integration time for signal search in ms. [`20.0`]
- `-sys sys[,...]`
  - Navigation system selection. Each character is one of `G` (GPS), `E` (Galileo), `J` (QZSS), `C` (BeiDou). [`G`]
- `-verb`
  - Enable verbose progress output to stdout.

- `-w file`
  - FFTW wisdom file. [`../python/fftw_wisdom.txt`]
- `-nav file`
  - RINEX navigation data file (required).
- `-out file`
  - Output solution file in RTKLIB solution format. [stdout]
- `-v`
  - Print version and exit.
- `file`
  - Input digital IF data file.

pocket_calib - Antenna array attitude and per-CH bias calibration

Synopsis

```
pocket_calib [-ts time] [-te time] [-f freq] [-v]
             -g geom_file -n nav_file -o out_file
             obs1 obs2 [obs3 ...]
```

Description

Estimate antenna-array hardware delays (per-element bias relative to CH1) and array attitude (roll / pitch / yaw) by per-epoch EKF using per-element RINEX OBS files and a RINEX NAV file. The receiver position is first estimated from CH1 obs by single-point positioning. The array geometry is read from a body-frame text file. Calibration results are written to a text file consumable by `pocket_trk` / `pocket_sdr.py` for live beam-forming.

Options ([]: default)

- `-ts time`, `-te time`
  - Start / end time (GPST, `YYYY/MM/DD HH:MM:SS`) to use from the OBS files. [all epochs]
- `-f freq`
  - Frequency index used for calibration (0 = L1, 1 = L2, ...). [`0`]
- `-g geom_file`
  - Array geometry file. One `x y z` (m) per line in body frame (X = right, Y = forward, Z = up). The k-th line is the position of CH(k); the first line is CH1 (reference, must be `0 0 0`).
- `-n nav_file`
  - RINEX navigation data file.
- `-o out_file`
  - Output text file. Stores the estimated `roll/pitch/yaw` and per-CH bias.
- `-v`
  - Print version and exit.
- `obs1 obs2 [obs3 ...]`
  - Per-element RINEX OBS files. The k-th file is the obs file for CH(k). At least 2 elements are required (typical: 4 or 8).

convbin - Convert receiver logs / RTCM streams to RINEX

Synopsis

```
convbin [option ...] file
```

Description

Convert receiver-binary log files, RTCM streams, and RINEX files to RINEX OBS / NAV / SBAS files. Pocket SDR's `convbin` is the RTKLIB `convbin` with extensions for RTCM3 array-mode conversion, MSM signal-ID extensions, and a default RINEX version of 3.05.

The full option list is long; only the most-used options are summarized below. For the complete option list and per-format message lists, run `convbin -h`.

Common Options

- `-r format`
  - Input format. One of `rtcm2`, `rtcm3`, `nov`, `oem3`, `ubx`, `ss2`, `hemis`, `stq`, `javad`, `nvs`, `binex`, `rt17`, `sbf`, `rinex`. Auto-detected from the file extension if omitted.
- `-v ver`
  - Output RINEX version (e.g. `3.05`, `3.04`, `2.11`). [`3.05`]
- `-ts y/m/d h:m:s`, `-te y/m/d h:m:s`, `-ti tint`
  - Start time, end time, observation interval (s).
- `-f n`
  - Number of signal frequencies to output (1..5 → L1..L5). [`5`]
- `-y sys[,...]`
  - Excluded systems, comma-separated (`G`/`R`/`E`/`J`/`S`/`C`/`I`).
- `-mask sig[,...]`
  - Signal/code masks. Each `sig` is `<sys>L<code>` (e.g. `GL1C`, `EL1B`, `JL2X`). `-nomask` clears specific masks.
- `-o file`, `-n file`, `-g file`, `-h file`, `-q file`, `-l file`, `-b file`, `-i file`, `-s file`
  - Output file paths for OBS, GPS NAV, GLO NAV, SBAS NAV, QZS NAV, GAL NAV, BDS NAV, NavIC NAV, and SBAS message files.
- `-d dir`
  - Output directory.
- `-trace level`

    - Enable RTKLIB trace at the given level.
- `-array`
    - (Pocket SDR extension) RTCM3 array mode. Splits MSM by station ID into per-CH RINEX OBS files. Requires `-r rtcm3`.
- `-xd`, `-xs`
    - (Pocket SDR extensions) Disable Doppler / SNR fields in the output OBS.
- `-h`
    - Print full help (extensive) and exit.
- `-ver`
    - Print version and exit.
- `file`
    - Input file path. Wildcards are *not* expanded on Windows.

str2str - Stream converter / forwarder (RTKLIB-derived)

Synopsis

```
str2str [-in stream[#format]] [-out stream[#format] ...] [options]
```

Description

Console version of the RTKLIB stream server. Reads from one input stream and forwards to multiple output streams. Optionally converts between formats (e.g. raw → RTCM3) using `-msg`. If `-in` or `-out` is omitted, stdin or stdout is used. To stop, send Ctrl-C (or `SIGINT` for a background process).

The full option list is long; only the most-used options are summarized below. For the complete list, run `str2str -h`.

Stream Path Forms

- `serial://port[:brate[:bsize[:parity[:stopb[:fctr]]]]][#port]`
- `tcpsvr://:port`
- `tcpcli://addr[:port]`
- `ntrip://[user[:passwd]@]addr[:port][/mntpnt]`
- `ntrips://[:passwd@]addr[:port]/mntpnt[:str]` (output only)
- `ntripc://[user:passwd@][:port]/mntpnt[:str]` (output only, caster)
- `[file://]path[::T][::+start][::x<speed>][::S=swap]` (file; `T` = time-tag mode, `+start` = start offset (s), `x<speed>` = replay speed, `S=swap` = swap interval (h))

Format Tags

`#rtcm2`, `#rtcm3`, `#nov`, `#oem3`, `#ubx`, `#ss2`, `#hemis`, `#stq`, `#javad`, `#nvs`, `#binex`, `#rt17`, `#sbf`.

Common Options

- `-in stream[#format]`
  - Input stream path with optional format tag.
- `-out stream[#format]`
  - Output stream path with optional format tag. May be repeated up to 4 times.
- `-msg "type[(tint)][,type[(tint)]...]"`
  - RTCM message types and per-type output intervals (s).

- `-sta id`
  - Station ID.
- `-opt opt`
  - Receiver-dependent options.
- `-s msec`, `-r msec`, `-n msec`, `-f sec`, `-b str_no`
  - Timeout (ms), reconnect interval (ms), NMEA request cycle (ms), file swap margin (s), back-relay stream number.
- `-c file`, `-c1 file` ... `-c4 file`
  - Input / per-output command file.
- `-p lat lon hgt`, `-px x y z`, `-a antinfo`, `-i rcvinfo`, `-o e n u`
  - Station position (geodetic / ECEF), antenna info, receiver info, antenna offset (ENU).
- `-l dir`, `-x proxy`
  - FTP/HTTP local directory; HTTP/Ntrip proxy.
- `-t level`, `-fl file`
  - Trace level; trace log file path.
- `-h`
  - Print full help and exit.
- `-v`, `-ver`
  - Print version and exit.